The Lesson

The problem with how most people use AI

Right now, most people open Claude or ChatGPT, type something, get a response, and start over. Maybe they save a good prompt somewhere and paste it in again next time. Maybe they hit the token limit and have to start a new conversation. Maybe they spend half the message just re-explaining who they are and what they are working on.

That works for quick questions. It falls apart the moment you try to do real, ongoing work. You are burning tokens on stuff that does not matter. You cannot edit what the AI produces at each step. And every conversation starts from zero.

The folder structure fixes all of that.

What tokens are and why this matters

Before we get into the structure, you need to understand one thing: tokens.

A token is roughly three quarters of a word. Sometimes a single word. Sometimes a long word like "hamburger" is three tokens. The term comes from NLP research in the 1990s. Researchers needed a unit smaller than a word because language does not break the same way across every language. They borrowed "token" from linguistics, which borrowed it from old English "taken," meaning a sign or a symbol. A token is just the smallest meaningful chunk of a sentence.

There are only so many tokens an AI can hold in its context window before it starts failing. When people say "context window," they mean how many tokens the AI can see at once. That window is finite.

So if you dump everything into one conversation, an AI writing a blog post is also reading your video production notes. You are wasting tokens on information that has nothing to do with the task. The folder structure solves this by separating your work into areas and only loading what is needed for the task at hand.

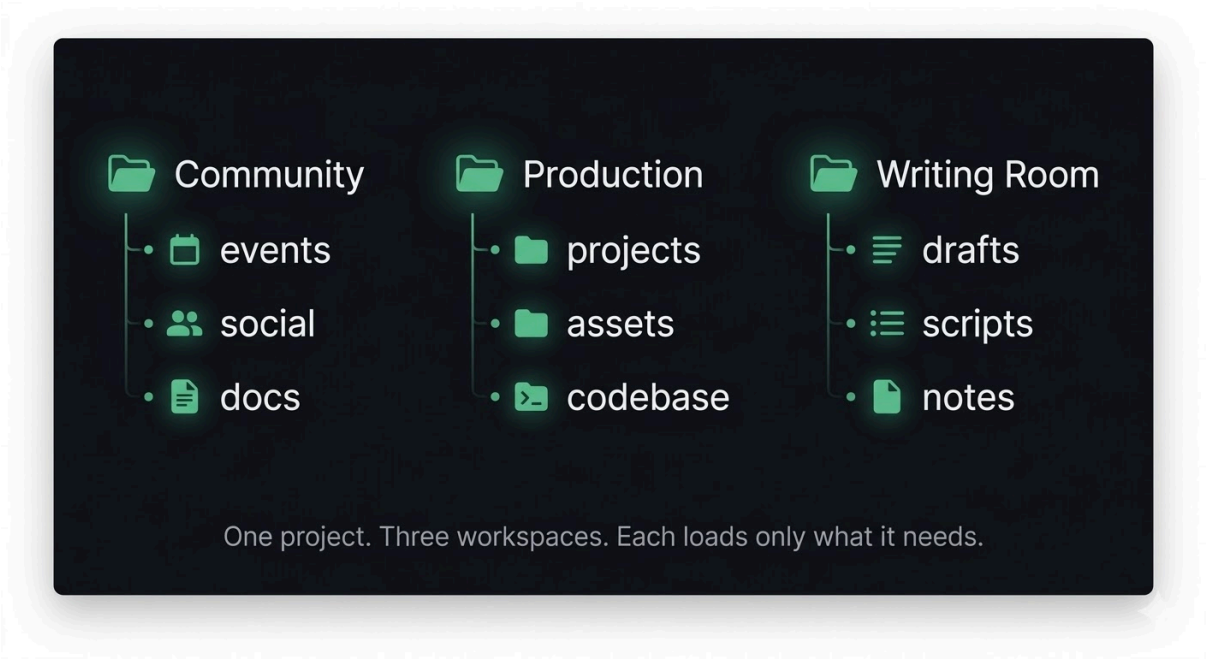
The workspace blueprint: three workspaces for three kinds of work

Here is the concept. Instead of one giant conversation or one massive file, you break your work into separate workspaces. Each workspace handles a different kind of work.

For example:

- **Community** — content, docs, community management
- **Production** — scripts, animations, code, builds
- **Writing Room** — blog posts, newsletters, thinking, client work

These are just examples. Your workspaces will look different depending on what you do. A freelancer might have Client Intake, Delivery, and Admin. A developer might have Frontend, Backend, and Docs. The names change. The structure stays the same.



What VS Code is (and why you do not need it)

In the video, you will see me working inside VS Code. VS Code is an IDE, a developer environment that lets you see all your folders and files in a sidebar and open them without double-clicking through your file system. It makes it easier to bounce between files.

You do not need VS Code to do any of this. You can work inside your regular file explorer, create folders, create text files, and edit them in Notepad or any text editor. You can also use Claude Cowork, which operates inside folders the same way. VS Code is just cleaner for managing lots of files. If it feels overwhelming, skip it for now and use whatever you are comfortable with.

What markdown is and why we use it

Every file in this system is a markdown file (.md). If you have never seen one before, markdown is just a text file with some lightweight formatting.

- Dashes make bullets
- Hashtags (pound signs) make headers
- Asterisks make **bold** or *italic*

That is it. When you talk to Claude and see its responses with bolding, headers, and clean formatting, it is already writing in markdown. Copy a Claude response and paste it into a plain text file. All the formatting turns into markdown symbols. That is how it was making it look that way.

A man named John Gruber created markdown in 2004. The idea was: write something readable as plain text that can also render into a formatted document. He named it "markdown" as a play on "markup language" (like HTML). Markdown strips away all the tags and complexity and makes it look like plain English with a few symbols.

Claude reads markdown cleanly. That is why we use it for every file in the system.

The three layer routing system

This is the core of the whole thing. Three layers. Each one has a job.

Layer 1: The Map (CLAUDE.md)

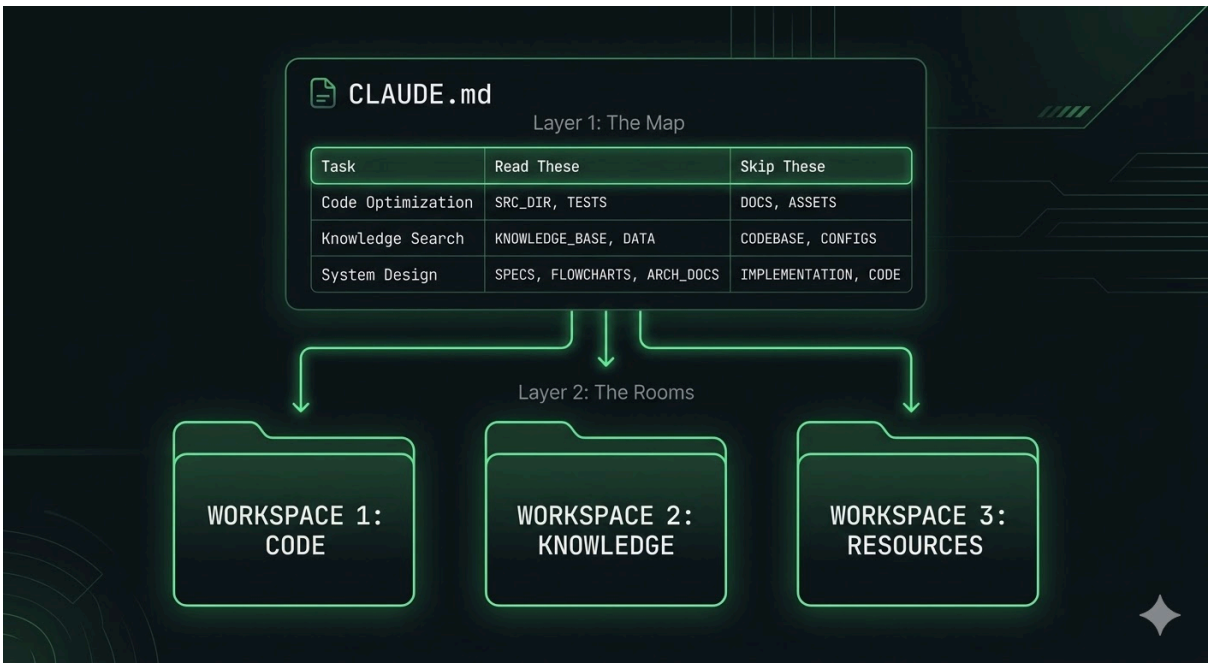
This is the top-level file. It sits at the root of your project folder. Every time Claude enters this workspace, it reads this file first.

Think of it as the floor plan. You walk into any building, the floor plan is on the wall, and you know where to go. The CLAUDE.md file tells the AI:

- What this project is
- What the folder structure looks like
- Naming conventions for files
- Where things go

This is the most important pattern in the whole system. Inside it, you put a simple table that tells the AI: for this task, read these files, skip those files, you might need these skills.

Without this, the AI either reads everything and wastes tokens, guesses wrong about what matters, or produces work you cannot edit along the way. This table eliminates all of those problems.



Layer 2: The Rooms (Workspace Context Files)

Each workspace has its own context file. When you tell Claude to work in the Writing Room, it reads the Writing Room context file. When you are in Production, it reads the Production context file. It only loads what it needs for where it is.

These context files describe:

- What this workspace is for
- What the process looks like (first I do this, then I do that)
- What files are in here and how they are organized
- What skills or tools to use in this workspace

You can write these by hand or have Claude help you write them. They are plain English. Short documents. A few paragraphs.

In the video, you can see what happens with almost no prompting at all. You say "go to writing room, let's start making something" and the AI immediately reads the context file, understands the workspace, loads the right voice and style, and asks what you want to build. One prompt. That is the power of Layer 2.

1. **Writing Room** — Blog posts, tutorials, docs. Has its own voice guide, style rules, and audience profiles.
2. **Production** — Builds demos, videos, and sample apps. Runs a 4-stage pipeline: briefs → specs → builds → output.
3. **Community** — Social posts, newsletters, event content. Repurposes material from the other two workspaces.

The key design idea: an AI agent drops into a workspace, reads its `CONTEXT.md` to understand what to do, does the work, and exits. Each workspace is intentionally siloed to manage token/context usage — you only load what's relevant to the current task.

It also defines:

- **Naming conventions** for files (e.g., `api-auth-guide-draft.md`)
- **File placement rules** for where content goes at each stage
- **Cross-workspace flow** (writing → production, both feed → community)
- **Tool/skill mappings** — which Claude Code skills and MCP tools are used in which workspace

Layer 3: The Tools (Skills, MCP Servers, and Plug-and-Play)

Layer 3 is where skills and tools live. Skills are processes that someone figured out and packaged into a set of files that tell Claude how to do a specific thing. A PowerPoint skill. A humanizer skill. A doc co-authoring skill.

The key here: you do not load every skill into every workspace. You wire skills into the workspaces where they are needed. Your Production workspace might reference a front-end design skill and a web app testing skill. Your Writing Room might reference a humanizer skill and a doc co-authoring skill.

You can reference 15, 20, or 100 skills in a project, but each workspace only loads the ones it needs. That is the plug-and-play idea. And if you need Claude to find a new skill or create one, you can wire that ability in too.

Naming conventions that replace databases

One more thing that makes this system work without any code or databases. In your CLAUDE.md, you add naming conventions.

If a blog draft is created, it gets named like: `api-auth-guide_draft.md` If it is a newsletter, it gets named like: `2026-03-launch-week.md` If it is a demo script version 2, it gets named like: `demo_v2.md`

The AI knows how to find, organize, and move files because the naming tells it everything. You can say "pull my demo v2 and build a spec from it" and Claude knows exactly where to look, what to pull, and what to do next. No SQL. No vector database. No Python injection. Just naming conventions.

How to make this yours

The template in the video uses a fake project with fake blog posts and demo scripts. That is intentional. You swap the names and rewrite the context files for your own work. The layers stay the same.

If you are a content creator:

- Writing Room → Script Lab
- Production → Edit Bay
- Community → Distribution Hub

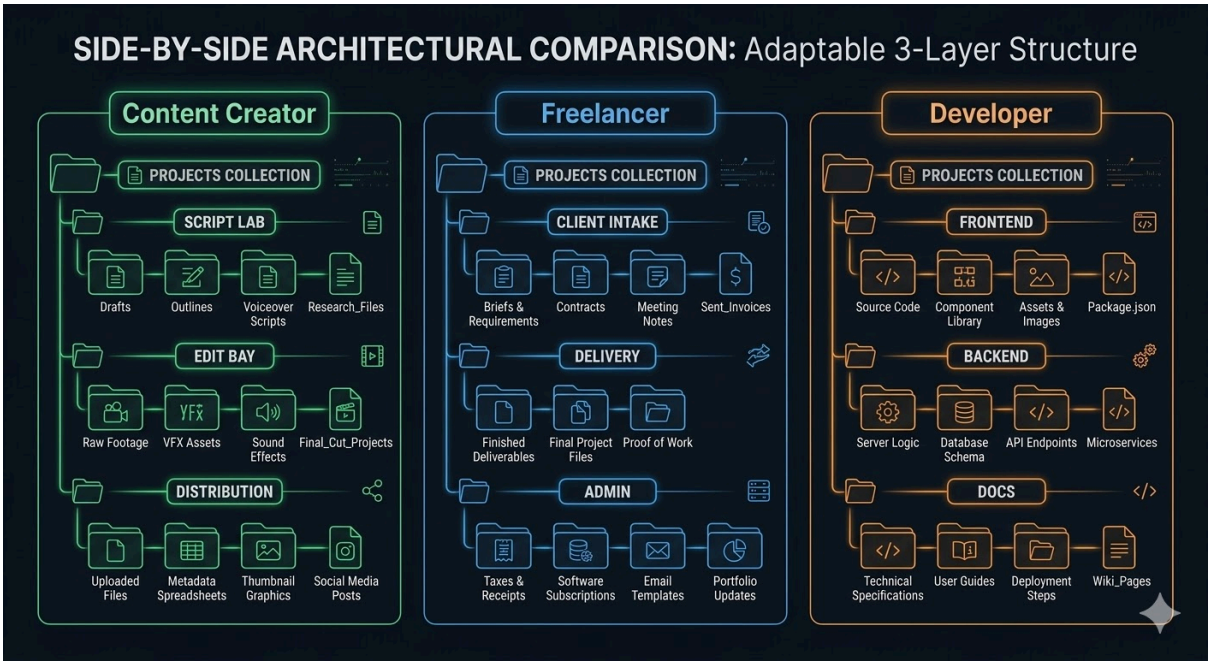
If you are a freelancer:

- Swap workspaces for: Client Intake, Delivery, Admin
- Your context files describe your client process instead of content production

If you are a developer:

- Swap workspaces for: Frontend, Backend, Docs
- Wire in the skills you actually use (testing, deployment, code review)

The three-layer routing system (map → rooms → tools) works the same way no matter what you do. You change the labels and the context. The architecture holds.



The research behind this

This is not a prompt trick. The thinking behind it comes from decades of software engineering, all the way back to 1972. Separation of concerns. Modular composition. The

idea that systems work better when each part does one job and the routing between parts is clear.

There is a full research paper on this if you want to go deep. It traces the history from early programming principles through to modern AI workflows and lays out a five-layer architecture (this video covers the three most important layers). You can download it and give it to Claude to explain it to you.

Research Paper: [ICM PAPER]

Quick start: your first three files

If you want to start right now with the simplest possible version, here it is. Make a folder. Put three files in it.

CLAUDE.md — Tell Claude who you are, what this project is, and where things go.

```
# Identity
You are helping [YOUR NAME] with [WHAT YOU DO].

# Folder Structure
- /drafts — work in progress
- /final — finished outputs
- /references — background material

# Rules
- Read this file first on every new task
- Ask before creating files outside of /drafts
- When unsure, ask
```

CONTEXT.md — Describe the current project.

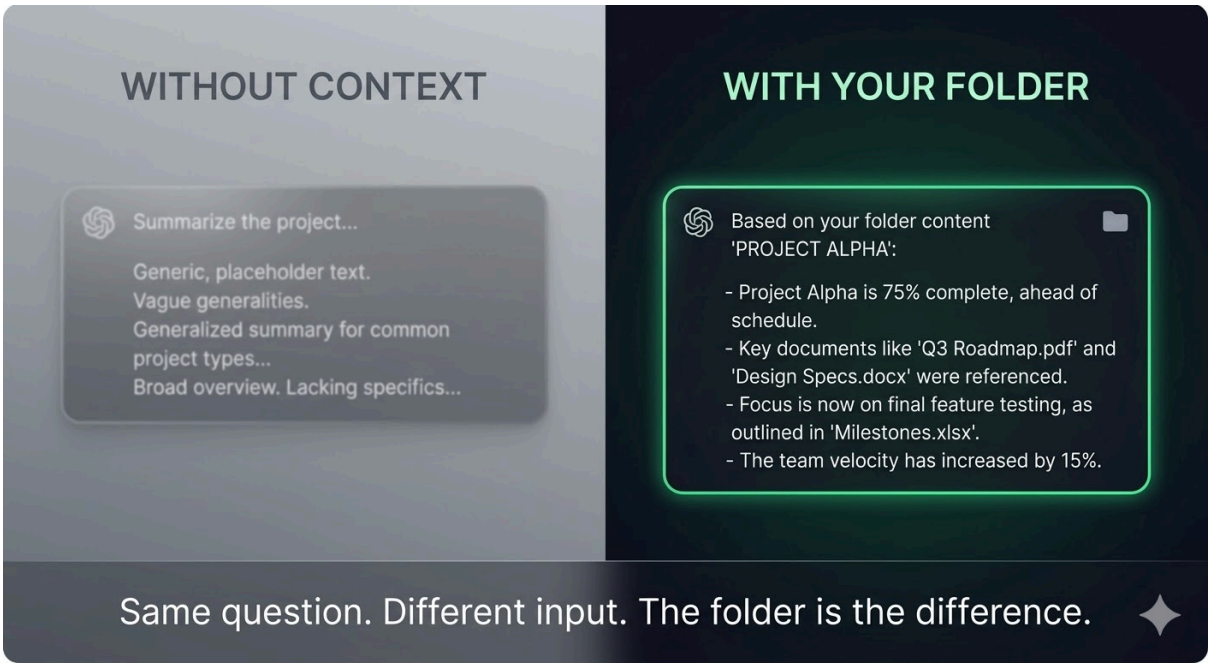
```
# Current Project
[What you are building or working on. 2-3 sentences.]

# What good looks like
[What a successful output looks like.]

# What to avoid
[Common mistakes or things you do not want.]
```

REFERENCES.md — Background material Claude should know about.

```
# References
```

[Examples, links, style guides, notes. Anything Claude should have access to but does not need to act on directly.]

Point Claude at the folder (using Claude Code, or upload the files to a Claude Project, or paste the contents into your first message). Ask it something related to your project. Notice the difference.



Go Deeper

- Want to customize this for your specific use case? **3.2: Customizing for Your Use Case** walks through three real examples.
- Want to understand *why* structured context changes AI output at a fundamental level? **2.3: How a 1953 Word Game Explains AI Memory** traces it back to Mad Libs.
- The concept of "value lives above whatever just got commoditized" shows up in **2.4: The Ladder That Explains Every AI Failure**.
- Ready to see this running in the terminal? **4.1: Install and First Use** gets you into Claude Code.
- **Premium** **and** **VIP members**: the full Workspace Blueprint template (the one from the video) is in **The Vault**.

3.2 Customizing for Your Use Case

What You'll Get From This

You will see how the three-layer folder architecture changes shape for three different kinds of work. The layers stay the same. The names, the context files, and the routing all change. You will walk away knowing how to build yours.

The Lesson

One structure, every use case

In **Section 3.1** you saw the full architecture: a top-level identity file (CLAUDE.md), workspace-level context files, and skills or tools that plug in where needed. The example in the video used a fake project with a community workspace, a production workspace, and a writing room.

That example was intentional. It is close enough to real work that you can see the logic, but generic enough that you have to make it your own. This lesson shows you how.

The principle is short: the layers do not change. The labels do. Your CLAUDE.md still sits at the top and routes everything. Each workspace still has its own context file. Skills still plug in where they are needed. But what each workspace is called, what the context files say, and what skills are wired in will look completely different depending on what you actually do.

Here are three real examples. Find the one closest to your situation, study it, then build your own.

Example 1: Content Creator

You make videos, write posts, manage a social presence, and probably do it mostly alone or with a very small team. Your work moves through a cycle: come up with ideas, write scripts or outlines, produce the content, publish it.

Your workspaces:

```
my-content-project/
├── CLAUDE.md
├── script-lab/
│   ├── CONTEXT.md
│   ├── ideas/
│   ├── drafts/
│   └── final/
├── production/
│   ├── CONTEXT.md
│   ├── briefs/
│   ├── specs/
│   ├── builds/
│   └── output/
└── distribution/
    ├── CONTEXT.md
    ├── platforms/
    ├── scheduling/
    └── analytics/
```

What each workspace does:

🟠 **Script Lab** — This is where thinking happens. Ideas go in. Drafts come out. The CONTEXT.md in this folder describes your voice, your audience, the kind of content you make, and the process you follow from idea to finished script. (could also reference separate MD files, if there is a lot of info in these) If you have a style guide or a list of topics you keep coming back to, that goes in here. Claude reads this context and writes in your voice from the first line.

🟡 **Production** — This is where content gets built. If you are making animations (like the pipeline in **Section 2.6**), this is where your briefs, specs, and build files live. If you are making simple videos, this might be where your shot lists, thumbnails, and description templates go. The CONTEXT.md here describes your production process, your tools, your visual standards.

🟢 **Distribution** — This is where finished content goes out. Platform-specific formatting (what works on Instagram vs LinkedIn vs YouTube), scheduling, repurposing long content into short clips. The CONTEXT.md describes your

platforms, posting cadence, and any rules about how content should be adapted per channel.

What the CLAUDE.md looks like:

Your top-level file tells Claude what this project is and how to route between the three workspaces. Something like:

```
# My Content Project

I create [TYPE OF CONTENT] for [AUDIENCE].

## Workspaces
- /script-lab — Idea development, writing, drafts
- /production — Building and producing content
- /distribution — Publishing, scheduling, repurposing

## Routing
| Task | Go to | Read |
|------|-------|------|
| Write or brainstorm | /script-lab | CONTEXT.md |
| Build or produce | /production | CONTEXT.md |
| Publish or repurpose | /distribution | CONTEXT.md |

## Naming conventions
- Drafts: topic-name_draft.md
- Final scripts: topic-name_final.md
- Published: YYYY-MM-platform-topic.md
```

Example 2: Freelancer / Consultant

You work with multiple clients. Each engagement has a lifecycle: intake, scoping, delivery, follow-up. You need Claude to shift between clients without bleeding context. You also need your own internal workspace for business development, templates, and admin.

Your workspaces:

```
my-consulting-practice/
├── CLAUDE.md
├── client-alpha/
│   ├── CONTEXT.md
```

```
│   ├── intake/
│   ├── deliverables/
│   └── communications/
├── client-beta/
│   ├── CONTEXT.md
│   ├── intake/
│   ├── deliverables/
│   └── communications/
├── templates/
│   ├── CONTEXT.md
│   ├── proposals/
│   ├── reports/
│   └── frameworks/
└── business-dev/
    ├── CONTEXT.md
    ├── pipeline/
    ├── outreach/
    └── case-studies/
```

What each workspace does:

🟠 **Client workspaces** (one per client) — Each client gets their own folder with their own CONTEXT.md. That file describes who the client is, what the engagement is, what phase you are in, what the deliverables are, and any client-specific rules (tone of voice, terminology they use, things to avoid). When you tell Claude "let's work on Alpha," it reads Alpha's context and nothing from Beta. No bleed. No confusion.

🟡 **Templates** — Your reusable frameworks. Proposal templates, report structures, analysis frameworks. The CONTEXT.md here describes what each template is for and how to use it. When you start a new engagement, you pull from templates into the client folder and customize.

🟢 **Business Dev** — Pipeline tracking, outreach drafts, case studies from past work. The CONTEXT.md describes your ideal client, your services, your positioning. When you need Claude to help draft an outreach email or build a case study, it reads this context.

What the CLAUDE.md looks like:

```
# My Consulting Practice
```

```
I am a [TYPE] consultant working with [TYPES OF CLIENTS].

## Active Clients
- /client-alpha — [One-line description of engagement]
- /client-beta — [One-line description of engagement]

## Internal
- /templates — Reusable proposals, reports, frameworks
- /business-dev — Pipeline, outreach, case studies

## Routing
| Task | Go to | Read |
|------|-------|------|
| Client work for Alpha | /client-alpha | CONTEXT.md |
| Client work for Beta | /client-beta | CONTEXT.md |
| Build a new proposal | /templates | CONTEXT.md, then client folder |
| Outreach or pipeline | /business-dev | CONTEXT.md |

## Rules
- Never reference one client's information in another client's workspace
- Proposals always start from /templates and get customized in the client
folder
- Deliverables go in /client-[name]/deliverables, drafts stay in working
folders
```

The key for freelancers: the client folders multiply. When you onboard a new client, you copy the structure, write a new CONTEXT.md, and Claude is ready. The CLAUDE.md at the top gets one new line in the routing table. That is it.

Example 3: Developer

You build software. You might work on one project or several. Your work involves planning, writing code, testing, deploying, and documenting. You probably already have opinions about folder structure. The difference here is that Claude reads it.

Your workspaces:

```
my-app/
├── CLAUDE.md
├── planning/
│   ├── CONTEXT.md
│   ├── specs/
│   ├── architecture/
│   └── decisions/
```

```
├── src/
│   ├── CONTEXT.md
│   ├── components/
│   ├── services/
│   ├── utils/
│   └── tests/
├── docs/
│   ├── CONTEXT.md
│   ├── api/
│   ├── guides/
│   └── changelog/
└── ops/
    ├── CONTEXT.md
    ├── deploy/
    ├── monitoring/
    └── scripts/
```

What each workspace does:

🟠 **Planning** — Specs, architecture decisions, design docs. The CONTEXT.md describes the app, the tech stack, the current priorities, and any architectural principles you follow. When you tell Claude to help you spec a new feature, it reads this context and works within your existing architecture.

🟡 **Src** — The actual codebase. The CONTEXT.md here describes the code structure, naming conventions, patterns you use (and patterns you avoid), testing requirements, and any libraries or frameworks that are standard in the project. Claude writes code that fits your codebase because it read the rules first.

🟢 **Docs** — API documentation, user guides, changelogs. The CONTEXT.md describes your documentation standards, the audience for each type of doc, and how docs relate to the code.

🔵 **Ops** — Deployment, monitoring, operational scripts. The CONTEXT.md describes your infrastructure, deploy process, and any runbook conventions.

What the CLAUDE.md looks like:

```
# My App

[APP NAME] — [One sentence description]
```

```
## Tech Stack
- Frontend: [framework]
- Backend: [language/framework]
- Database: [type]
- Deploy: [platform]

## Workspaces
- /planning — Specs, architecture, decisions
- /src — Application code
- /docs — Documentation
- /ops — Deployment and operations

## Routing
| Task | Go to | Read | Skills |
|------|-------|------|--------|
| Spec a feature | /planning | CONTEXT.md | — |
| Write code | /src | CONTEXT.md | testing-skill |
| Write docs | /docs | CONTEXT.md | doc-authoring-skill |
| Deploy or debug | /ops | CONTEXT.md | — |

## Naming conventions
- Specs: feature-name_spec.md
- Components: PascalCase
- Tests: feature-name.test.ts
- Decision records: YYYY-MM-DD-decision-title.md
```

Developers will notice the routing table has a Skills column. This is Layer 3 from 3.1. You can wire testing skills, documentation skills, or code review skills into specific workspaces so they only load when relevant. Your planning workspace does not need the testing skill. Your src workspace does

---

How to build yours

You do not need to match any of these exactly. The process is the same regardless of what you do.

Step 1: List your workspaces. Think about the 2-4 major areas of your work. What are the modes you shift between? Writing and building are different workspaces. Client A and Client B are different workspaces. Planning and executing might be

different workspaces. If you find yourself wishing Claude would "forget" what it was just doing and focus on something else, that is a workspace boundary.

Step 2: Write a CONTEXT.md for each one. Describe what happens in this workspace, what the process is, what files live here, and what good work looks like. Keep it under a page. You can always add more later.

Step 3: Write your CLAUDE.md. List the workspaces and build the routing table. For each type of task, tell Claude where to go and what to read. Add naming conventions so Claude knows how to organize files.

Step 4: Start working. Point Claude at the folder and give it a task. See what happens. Adjust the context files based on what Claude gets right and what it gets wrong. The first version will not be perfect. It will be better than no structure at all, and it will get better every time you edit a context file.

The context files are living documents. Edit them as your projects change, as you learn what Claude needs to know, as you figure out what to cut. The people who get the most out of this system are the ones who treat the context files like working notes, not finished documents.

Resources

- **Premium and VIP members:** the full **Workspace Blueprint** and Example Production MD files are in **The Vault**

Go Deeper

- If you have not done the full walkthrough yet, **3.1: The Full Walkthrough** covers the three-layer system, naming conventions, and the research behind all of it.

- Common mistakes people make when setting up their folders are covered in **3.3: Common Mistakes and How to Fix Them**. Worth reading before you finalize your structure.
- The prompting framework from **1.3 How to Structure Any Prompt** works inside each workspace. Your context files handle identity and project context. Your prompts handle task, constraints, and output format. The two systems work together.
- The production pipeline example from the **2.6: Video as Code** lesson is a real implementation of the content creator structure above. If you want to see what a mature version of this looks like in action, start there.

3.3 Common Mistakes and How to Fix Them

What You'll Get From This

The seven mistakes people make most often when setting up the folder architecture, and how to fix each one before it wastes your time.

The Lesson

Every one of these comes from real community members. People post their setups, run into walls, and ask for help. The same problems show up over and over. If you read this before you finalize your structure from 3.2, you will skip most of them.

Mistake 1: Making the CLAUDE.md too long

The CLAUDE.md is a routing file. It tells Claude where things are and where to go. That is its job. It is not a project brief. It is not a style guide. It is not a brain dump of everything you want Claude to know.

When the CLAUDE.md gets too long, two things happen. First, Claude burns tokens reading information that is not relevant to the current task. Second, the signal gets

buried in noise. The routing instructions (the part that actually matters) get lost in paragraphs of background context.

The fix: Your CLAUDE.md should fit on one screen. Identity, folder structure, routing table, naming conventions. That is it. Everything else belongs in a workspace CONTEXT.md where it only gets loaded when Claude is actually working in that area. If your CLAUDE.md is longer than 40-50 lines, you have context files hiding inside it. Pull them out.

Mistake 2: Skipping the routing table

Some people set up the folders, write the context files, and never put a routing table in the CLAUDE.md. They assume Claude will figure it out.

Sometimes Claude does figure it out. But "sometimes" is the problem. Without a routing table, Claude has to guess which files to read and which to skip. It might read everything (wasting tokens) or read the wrong context file (getting confused about what workspace it is in). The output gets inconsistent. Some responses are great, some are off, and you cannot tell why.

The fix: The routing table does not need to be complicated. Three columns: task, where to go, what to read. One row per type of work. Claude reads the table, matches the task, goes to the right place. Every time.

```
| Task | Go to | Read |
|------|-------|------|
| Write content | /script-lab | CONTEXT.md |
| Build something | /production | CONTEXT.md |
| Publish or schedule | /distribution | CONTEXT.md |
```

If you added skills (Layer 3), add a fourth column. Otherwise, three is enough.

Mistake 3: Too many workspaces

Someone sets up eight workspaces for a project that really only has two or three modes of work. They have a workspace for research, a workspace for notes, a workspace for drafts, a workspace for editing, a workspace for final review, a workspace for publishing. Each one has its own CONTEXT.md. The CLAUDE.md has a routing table with twelve rows.

Now the overhead of maintaining the system is bigger than the work itself. Context files go stale. Claude spends more time navigating between workspaces than working in them. The whole point of the architecture was to reduce complexity, and instead it multiplied it.

The fix: Start with two or three workspaces. You can always add more. The question to ask: "Do I shift mental modes between these tasks?" Writing and building are different mental modes. That is two workspaces. Drafting and editing are the same mental mode at different stages. That is one workspace with a process inside it, not two workspaces.

If you are not sure whether something deserves its own workspace, it does not. Keep it as a subfolder inside an existing workspace. You can always split it out later if the work grows.

Mistake 4: Writing context files that describe what AI should be instead of what the work is

This one is subtle. People write context files full of instructions about Claude's personality. "Be creative. Be concise. Be professional. Think step by step. Use a warm tone." They spend 30 lines describing how Claude should behave and two lines describing the actual project.

Claude responds to context about the work far more than context about itself. Telling Claude "you are a senior copywriter" gives it a role. Telling Claude "the audience is mid-market HR directors who have tried three other tools and are skeptical of AI

claims" gives it something to actually work with. The second one changes the output more than the first.

The fix: Flip the ratio. Spend 80% of your context file describing the work: what the project is, who the audience is, what has already been done, what good output looks like, what to avoid. Spend 20% or less on behavioral instructions. If your context file reads like a personality quiz, rewrite it. If it reads like a project brief that a new team member could pick up and start working from, you are in the right place.

Mistake 5: Never updating the context files

Someone sets up the folder in week one. Writes the context files. Gets great results for two weeks. Then the project evolves. New requirements, new direction, new constraints. But the context files still say what they said on day one. The output starts drifting. Claude is still working from old context. The member thinks Claude "got worse" when really Claude is doing exactly what the context tells it to do. The context is just stale.

The fix: Treat context files like working notes. When the project changes, edit the context. When you learn something new about what Claude needs to know, add it. When a constraint no longer applies, remove it. This takes 30 seconds per edit. It is the single highest-leverage habit in the whole system.

Some people add a "Last updated" line at the top of each context file. Simple but effective. When you open a workspace and see "Last updated: six weeks ago," you know to review it before working.

Mistake 6: Putting everything in one folder and hoping the CLAUDE.md sorts it out

The opposite of too many workspaces. Everything lives in one flat directory. Fifty files. No subfolders. The CLAUDE.md tries to route between them using file names alone. Claude reads the whole directory listing, picks what it thinks is relevant, and often picks wrong.

This is the equivalent of dumping every document on one desk and asking someone to find the right one. They can do it. It just takes longer, they make more mistakes, and they get frustrated.

The fix: If you have more than 8-10 files at the same level, you need subfolders. Group by workspace (what kind of work), then by stage or type within the workspace. The folder structure is the architecture. It tells Claude what belongs together. Let it do that job.

Mistake 7: Building the whole system before using it

The most common mistake. Someone reads 3.1 and 3.2, gets inspired, and spends an entire weekend building a perfect folder architecture with six workspaces, detailed context files, naming conventions, skills wired into every workspace, and a routing table with twenty rows. They have not used Claude once during this process. They built the factory without ever making a product.

Then they start using it and realize half the decisions they made do not match how they actually work. The workspace boundaries are wrong. The context files describe what they thought they would need, not what they actually need. They have to rebuild.

The fix: Build the minimum. One CLAUDE.md, one or two workspaces, one CONTEXT.md per workspace. Start working. After a few days, you will know what is missing. Add it then. After a week, you will know what is wrong. Fix it then.

The best folder setups in the community were all built incrementally. They started simple and grew based on real work. The ones that stalled were the ones that tried to be complete before day one.

Your first version should take 15 minutes. If it took longer, you over-built.

The pattern across all seven

If you look at these together, they all point in the same direction. Keep the system small. Keep it focused on the work, not on Claude. Update it as you go. Let the structure grow from use, not from planning.

The folder architecture is powerful because it is simple. Folders and text files. The moment it starts feeling heavy or complicated, something went wrong. Go back to the three layers from 3.1: map, rooms, tools. If your CLAUDE.md is the map, your context files are the rooms, and your skills are the tools, and each one is doing only its job, the system stays clean.

Go Deeper

- If you have not built your structure yet, go back to **3.2: Customizing for Your Use Case** and start from one of the three examples. Build the minimum version. Come back here when you run into something.
- The prompting framework from **1.3 How to Structure Any Prompt** helps with Mistake 4. Your context files handle identity and project context. Your prompts handle the per-task stuff. Separating those two keeps your context files focused on the work.
- The production pipeline in **2.6: Video as Code** is an example of a system that grew incrementally. It started as a simple folder and became a full spec-to-animation pipeline over weeks of use. That is the right pace.
- **The Vault** (Premium) has the full Blueprints with example context files already written. If you want a starting point that is further along than the quick-start three files but still customizable, that is what it is for.